



CAHIER DES CHARGES TECHNIQUE

Assistant Pédagogique Intelligent (Quiz Bot)

Génération automatique de questionnaires à partir de supports de cours via RAG et API LLM

Mots-clés	RAG, API LLM, Programmation Web, Automatisation pédagogique, Python
Classe cible	Cycle Ingénieur — Niveau 1 ou 2
Durée du stage	1 à 2 mois
Configuration	1 étudiant
Projet	Projet de Stage d'Été 2025-2026

1. Contexte et Problématique

La création de questionnaires pédagogiques est une tâche chronophage pour les enseignants. Chaque semestre, ceux-ci doivent élaborer des QCM, des questions ouvertes et des exercices d'auto-évaluation pour leurs différents modules. Ce processus manuel, bien que nécessaire, limite la réactivité de l'enseignant et réduit la fréquence à laquelle les étudiants peuvent s'entraîner sur du contenu frais et pertinent.

Parallèlement, les avancées récentes en matière de grands modèles de langage (LLM) et de techniques de Retrieval-Augmented Generation (RAG) offrent des opportunités uniques pour automatiser cette tâche. Un système capable d'extraire les concepts clés d'un document de cours (PDF, diapositives) et de générer des questions pertinentes et variées permettrait de libérer du temps pédagogique tout en offrant aux étudiants un outil d'auto-évaluation accessible à tout moment.

Ce projet vise donc à concevoir et développer une application web, dénommée Quiz Bot, qui permet à un enseignant de téléverser un support de cours et de recevoir automatiquement des questionnaires générés par l'intelligence artificielle, basés exclusivement sur le contenu de ce document. Les étudiants pourront ensuite s'auto-évaluer et recevoir un feedback instantané.

2. Fonctionnement du Projet

Le système repose sur une architecture en trois couches principales : l'interface utilisateur, le backend de traitement et le moteur de génération basé sur un LLM.

Dans un premier temps, l'enseignant téléverse un fichier PDF ou un ensemble de diapositives (format PPTX) via l'interface web. Le backend procède alors à l'extraction du texte brut à partir du document à l'aide de bibliothèques Python telles que PyMuPDF ou python-pptx. Le texte extrait est ensuite découpé en morceaux (chunks) de taille adaptée, puis vectorisé à l'aide d'un modèle d'embedding et stocké dans une base de données vectorielle (par exemple ChromaDB ou FAISS).

Lorsque l'enseignant demande la génération d'un quiz, le système RAG interroge la base vectorielle pour récupérer les passages les plus pertinents du cours en fonction du type de questions souhaité (QCM, questions ouvertes). Ces passages contextuels sont ensuite injectés dans un prompt structuré envoyé à l'API d'un LLM (OpenAI GPT, Mistral, ou modèle open-source via Hugging Face). Le modèle génère les questions, les réponses attendues et les explications associées.

Côté étudiant, l'interface affiche le quiz généré sous forme interactive. L'étudiant répond aux questions, et le système compare ses réponses avec les réponses de référence pour fournir un feedback immédiat, un score et des explications détaillées. Les résultats peuvent être exportés en format PDF ou JSON pour intégration dans un espace numérique de travail (ENT).

3. Objectifs du Projet

3.1 Objectif principal

Concevoir une application web fonctionnelle permettant la génération automatique de questionnaires pédagogiques à partir de supports de cours, en utilisant les techniques RAG et les API de grands modèles de langage.

3.2 Objectifs secondaires

- Développer une interface utilisateur intuitive pour le téléversement de documents et la configuration des quiz.
- Implémenter un pipeline RAG complet : extraction de texte, chunking, embedding, stockage vectoriel et retrieval.
- Intégrer une API LLM pour la génération de QCM et de questions ouvertes pertinentes.
- Créer un module d'auto-évaluation avec feedback instantané pour les étudiants.
- Développer un système d'export des quiz en formats PDF et JSON.
- Rédiger un guide d'utilisation synthétique destiné aux enseignants.

4. Algorithmes et Technologies

Le projet s'appuie sur un ensemble de technologies modernes et bien documentées, adaptées au profil d'un étudiant en cycle ingénieur de niveau 1 ou 2.

Tableau 1 : Stack technique du projet

Composant	Technologies recommandées
Backend	Python 3.10+, Flask ou FastAPI, LangChain
Frontend	Streamlit, ou HTML/CSS/JS avec Flask
Extraction PDF	PyMuPDF (fitz), pdfplumber
Embeddings	sentence-transformers, OpenAI Embeddings
Base vectorielle	ChromaDB ou FAISS
LLM	OpenAI GPT-4 / GPT-3.5, Mistral API, Hugging Face Inference API
Export PDF	ReportLab, FPDF2
Export JSON	json (bibliothèque standard Python)
Versionnage	Git, GitHub

L'algorithme principal du système suit le paradigme RAG (Retrieval-Augmented Generation). Le

processus commence par le découpage du texte en segments de taille optimale (environ 500 à 1000 tokens par chunk), avec un chevauchement de 100 à 200 tokens pour éviter la perte d'information aux frontières. Chaque segment est ensuite transformé en vecteur numérique via un modèle d'embedding et indexé dans la base vectorielle. Lors de la génération d'un quiz, une requête sémantique est effectuée pour extraire les K segments les plus pertinents, qui servent de contexte au LLM pour produire des questions fidèles au contenu du cours.

5. Description du Système

Le système se décompose en trois modules principaux, chacun correspondant à un acteur ou un cas d'utilisation spécifique du projet.

5.1 Module Enseignant

Ce module permet à l'enseignant de téléverser un ou plusieurs fichiers de cours (PDF, PPTX). Après le téléversement, le système procède automatiquement à l'extraction, au découpage et à l'indexation du contenu. L'enseignant peut ensuite configurer les paramètres du quiz : nombre de questions, type de questions (QCM, questions ouvertes, ou mélange), niveau de difficulté et thèmes spécifiques à couvrir. Une fois le quiz généré, l'enseignant peut le prévisualiser, le modifier manuellement, puis le publier pour les étudiants ou l'exporter en PDF/JSON.

5.2 Module Étudiant

L'étudiant accède aux quiz publiés via une interface simple et claire. Pour chaque question, il sélectionne ou saisit sa réponse, puis valide. Le système calcule immédiatement le score, affiche les réponses correctes et fournit des explications détaillées pour chaque question. L'étudiant peut également consulter un résumé de sa performance avec des statistiques par thème et par type de question.

5.3 Module d'Export

Ce module gère la conversion des quiz générés vers des formats standards. L'export PDF produit un document formaté, prêt à être imprimé ou intégré dans un ENT. L'export JSON fournit une représentation structurée des questions, réponses et métadonnées, facilitant l'intégration avec d'autres plateformes pédagogiques ou systèmes de gestion de l'apprentissage (LMS).

6. Cycle de Vie du Projet

Le projet suit un modèle de développement itératif et incrémental, adapté à la durée contrainte d'un stage d'été de 1 à 2 mois. Ce modèle permet de livrer des fonctionnalités progressivement tout en intégrant les retours de l'encadrant à chaque itération.

La première semaine est consacrée à l'installation de l'environnement de travail, à la prise en main des technologies (Flask/FastAPI, LangChain, bibliothèques d'extraction) et à la définition précise des

spécifications fonctionnelles. Les semaines suivantes sont organisées en sprints de deux semaines : le premier sprint porte sur le pipeline d'ingestion des documents et le stockage vectoriel, le deuxième sur l'intégration de l'API LLM et la génération de questions, le troisième sur l'interface étudiant et l'auto-évaluation, et le dernier sur l'export, les tests et la rédaction du guide d'utilisation. Chaque sprint se termine par une démonstration et une phase de correction.

7. Exigences du Système

7.1 Exigences matérielles

- Ordinateur portable ou poste de travail avec processeur i5 ou supérieur
- 8 Go de RAM minimum (16 Go recommandés pour le traitement de gros documents)
- Connexion Internet stable et fiable (nécessaire pour les appels API LLM)
- Espace disque de 10 Go minimum pour l'environnement de développement et les données

7.2 Exigences logicielles

- Système d'exploitation : Windows 10/11, macOS 12+ ou Ubuntu 20.04+
- Python 3.10 ou supérieur
- Éditeur de code : Visual Studio Code ou PyCharm
- Navigateur web moderne : Chrome, Firefox ou Edge
- Git et compte GitHub pour le versionnage du code
- Clé API pour le service LLM choisi (OpenAI, Mistral ou Hugging Face)

8. Livrables Attendus

À l'issue du stage, l'étudiant doit fournir l'ensemble des livrables suivants, qui constituent la base d'évaluation du travail réalisé.

Tableau 2 : Liste des livrables

Livable	Description
Application Web	Application fonctionnelle avec interface enseignant et étudiant
Code source	Dépôt GitHub avec documentation du code et README
Export PDF/JSON	Système d'export opérationnel pour les deux formats
Guide d'utilisation	Document synthétique à destination des enseignants
Rapport de stage	Rapport complet décrivant le travail réalisé et les choix techniques

9. Limites et Contraintes

- La qualité des questions générées dépend directement de la qualité du document source : un PDF mal structuré ou contenant principalement des images non OCRisées produira des résultats inférieurs.
- Les appels à l'API LLM nécessitent une connexion Internet permanente et engendrent des coûts d'utilisation variables selon le fournisseur choisi.
- Le système ne prend pas en charge nativement les documents scannés ou les supports contenant des équations mathématiques complexes, qui nécessiteraient un module OCR supplémentaire.
- L'évaluation des questions ouvertes repose sur une comparaison sémantique qui peut ne pas capturer toutes les nuances des réponses des étudiants.

10. Références

1. [Lewis, P. et al. \(2020\). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.](#)
2. [LangChain Documentation. https://python.langchain.com/docs/](https://python.langchain.com/docs/)
3. [ChromaDB Documentation. https://www.trychroma.com/](https://www.trychroma.com/)
4. [OpenAI API Documentation. https://platform.openai.com/docs/](https://platform.openai.com/docs/)
5. [Streamlit Documentation. https://docs.streamlit.io/](https://docs.streamlit.io/)